`if, while, for`: More ways to build logic and control the flow in a program

Sudeep K S, Mahindra University (September 2026)

- 'If' statement:

Do something (execute statement or a sequence of statements) if a given condition is true.

```
if (condition)
{
    // sequence of statements
}
```

We can have an optional 'else' part:

```
if (condition)
{
    // sequence of statements
} else {
   // another sequence of statements
}
```

```
#include <stdio.h>

int main()
{
    int a, b, c;
    a = 10; b = 5; c = 2;

    if (a < b || c!= 0)
       a = a + b;
    else {
      a = a - b;
      c = 0;
    }

    printf("a is: %d\n", a);
    printf("b is: %d\n", b);
    printf("c is: %d\n", c);
    return 0;
}
```

- 'While' loop:

Keep doing (execute a statement or a sequence of statements) as long as a condition is true.

```
while (condition)
{
    // sequence of statements
}
```

Example:

```
int a, b, c;
a = 2; b = 10; c = -1;

while (a <= b) // true initially
{
   a = a + 2;
   printf("inside while\n");
   c = c + 2; // a becomes 12 at the end, that time a becomes bigger than b
}

printf("a is: %d\n", a);
printf("b is: %d\n", b);
printf("c is: %d\n", c);
```

What does this print?

Important: You need to make sure that the loop does not go on forever, as in the below example.

```
int a, b, c;
a = 2; b = 10; c = -1;

while (a < b) // true initially
{
   printf("inside while\n");
   c = c + 2; // this does not change a or b, so the condition remains true
}
```

- ‘For’ loop:

Keep doing (execute a statement or a sequence of statements) as long as a condition is true. We get to initialize and update within the header itself, unlike with the while loop.

```
for (initialize; condition; update)
{
    // sequence of statements
}
```

Example 1: A simple for loop. What does it print?

```
for (i = 0; i < 100; i++)
{
    printf("i is %d\n",)
}
```

First we initialize the variable; then check the condition and execute the block of statements if the condition is true; and after executing the statement (or the set of statements) once, do the update part. Then check the condition again, execute the block of statements, update.. It goes on like this.

Example 2: Check if a number is prime, with a for loop.

```
#include <stdio.h>
#include <stdbool.h>
int main()
{
    int i, n;
    bool flag = false;

    printf(“enter a positive integer number: \n”);
    scanf(“%d”, &n); // the number being tested. Example: 20189

    if (n == 1) {
       printf("n is neither PRIME nor COMPOSITE");
       return 0;
    }

    if (n == 2) {
       printf("n is PRIME");
       return 0;
    }
```

```
    // main check starts here
    for (i = 2; i < n; i++)
    {
        if (n % i == 0) {
            flag = true;
            printf("n is COMPOSITE and %d is a factor", i);
            break;
        }
    }

    if (flag == false)
       printf("n is PRIME"); // if it is prime

    return 0;
}
```

Example 3: Find n^{th} Fibonacci number, using a for loop. (Fibonacci series: 0, 1 , 1, 2, 3, 5, 8, ..)

```
#include <stdio.h>
int main()
{
    int a, b, sum, i, n;

    // Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, ..
    printf(“enter a positive integer n: \n”);
    scanf(“%d”, &n); // (example: n = 8 should output 13)

    a = 0; b = 1;

    for (i = 3; i <= n; i++)
    {
        sum = a + b;
        a = b;
        b = sum;
    }
    printf("number is %d", sum);
    return 0;
}
```